iacc instituto profesional

EVALUACIÓN

Evaluación de sistemas QA. Semana 5

Amparo Valdivia
27, febrero de 2026
Técnico en Análisis y Programación.

DESARROLLO:

La clínica oftalmológica "Visión Clara":

Nombre	Sistema_web_atencion.
Product Owner	Clínica oftalmológica Visión Clara.
Objetivo	Mejorar su atención al paciente mediante la implementación de un sistema web integral para consultas oftalmológicas.
Requerimientos técnicos	Este sistema permitirá a los pacientes: - Agendar citas de manera eficiente y rápida. - Acceder a su historial médico desde cualquier dispositivo con conexión a internet. - Recibir recordatorios automatizados de citas pendientes. - El personal médico tendrá la capacidad de revisar y actualizar los registros de los pacientes de manera ágil y precisa.

Metodologías.	**Metodología Ágil (Scrum/Kanban):** **Basado en Scrum,** organizar el equipo, definir responsabilidades, las reuniones diarias y que el trabajo se divida en "Sprints" de 2 a 3 semanas, siendo prioridad basado en las métricas de los nodos donde se produzcan mas errores y sean procesos críticos. Entregas rápidas y periódicas, se pretende lograr el avance para etapas de integración, e introducir pruebas automáticas desde etapas tempranas.
Sincronización DevOps/QA	Ejemplo, Si el equipo de QA necesita que cada vez que se suba código se corran 50 pruebas de Selenium, DevOps automatiza ese paso en Git. Si el equipo de QA detecta en **Jira** que hay muchos fallos en el agendamiento de Citas, piden a DevOps que refuerce las pruebas automáticas en ese módulo específico. **Uso de Gatekeeping, la Portería,** el equipo DevOps configurará Git para que

	no permita hacer Merge, unir el código si las pruebas unitarias realizadas con Pytest no están en verde. Así el flujo nunca se corrompe. **Triage de Fallas QA,** Al usar Jira para categorizar por gravedad y urgencia, el equipo QA se asegura de que los desarrolladores no paralicen el tiempo arreglando, por ejemplo, color de botón si el sistema de recetas médicas no se está guardando, la priorización en marcha, criterio de desarrollo, daily meeting. Como el alcance es que la app debe cargar en menos de **2 segundos**. Eso es un **KPI (Indicador Clave de Desempeño)** de rendimiento. • **Labor de DevOps:** Automatizar una prueba de estrés que simule 100 pacientes entrando al mismo tiempo, prueba de carga. • **Labor de QA:** Verificar que, si carga rápido, la información que muestra sea la correcta para el paciente.

Equipo DevOps	Se define cómo se entrega el software. Es la parte técnica que automatiza hasta el servidor. Se enfoca en herramientas y procesos automáticos, es el que guiará todo el proceso ya que se implementarán pruebas automatizadas en todo el proceso de vida del software, un Deep work automatizado, este equipo debe estar sincronizado con QA ya que se designaron responsabilidades para mantener el monitoreo constante y no romper el flujo continuo. En las pruebas unitarias configurar el repositorio (Git). Pruebas de integración, que se realice de forma continua.
Equipo QA	El proceso de reporte de fallas en el sistema de consultas oftalmológicas sería fundamental para garantizar la calidad del producto final, utilizando Jira.

	Establecer protocolos y herramientas para que los usuarios y el equipo de desarrollo puedan reportar cualquier anomalía o problema detectado durante el uso del sistema. - sistema de seguimiento de problemas. - categorizarían según su gravedad y urgencia. - analizaría y priorizaría estos reportes, - trabajando en conjunto con el equipo de desarrollo para resolver las fallas identificadas y asegurar la entrega de un sistema confiable y de alta calidad.
Alcances	La app debe cargar en menos de 2 segundos, Facilitar la gestión de la información clínica y mejorar la calidad de la atención oftalmológica ofrecida por la clínica.
Herramientas	**Pruebas de sistema y usabilidad, Selenium IDE,** extensión del navegador que graba los clics del usuario donde se puede analizar la forma en que se comporta el sistema al agendar el paciente una hora, grabar el proceso de Agendar

	Cita, realizar Play para ver si la web sigue funcionando bien y mantener el sitio. **Selenium WebDriver: Python** o **Java** para programar pruebas más complejas, por ejemplo, probar qué pasa si un paciente ingresa un rut inválido, revisar si se están actualizando los datos de los pacientes. **Pruebas Unitarias** JUnit para clases / PyTest, unittest Python, estas herramientas prueban funciones como calcular_disponibilidad(). **Pruebas de integración**, MockK / Mockito**CI ,** asegurarse que el servidor esté funcionando.

Este sistema permitirá a los pacientes:

Hice una lista de lo que revisaría para verificar que el código cumple con los requerimientos, recalco que son en base a pruebas unitarias y que automatizar estas pruebas agilizaría el proceso inicial ya que no hay código previo y hay que realizar las pruebas desde cero y estas son iniciales, así que, recomiendo en referencia a las pruebas unitarias subirlas a git para compartir con el equipo el proceso y realizar commit inmediatamente para evitar cambios que provoquen errores y que estos se puedan reversar, esto funciona como respaldo, realizar add, entregas periódicas en git, llevar el

control de errores en jira, manejar los procesos por separado y cumplir con entregas periódicas fundamentadas, para agilizar las integraciones y se puedan empezar a realizar pruebas de sistema lo antes posible, **"Shift Left Testing"** (empezar las pruebas lo más temprano posible en el desarrollo), que es un pilar fundamental de DevOps, si se detectan errores crear nuevos nodos y no interrumpir el flujo del sistema, es esta la razón que se recomienda que en etapas tempranas la automatización de las pruebas unitarias es esencial, también veo la necesidad de que se realicen varias de ellas, quiero decir saturar las probabilidades de fallo, **pruebas de borde** o casos de estrés, y compartir los hallazgos y documentarlos.

- Agendar citas de manera eficiente y rápida.

  Revisar que los datos de entrada tengan try except, para evitar errores y avisar al usuario si ocurrió un error y como debe ingresar sus datos personales.

  Prueba Unitaria: Crear un test que envíe un RUT con letras o una fecha pasada para asegurar que el try-except capture el error y no explote el sistema.

  Revisar que exista un sistema de login donde exista la autenticación de datos y validaciones como número y tipo de caracteres.

  Prueba Unitaria: Probar contraseñas vacías, muy cortas o sin caracteres especiales. Usar una función de validación que JUnit pueda chequear.

  Revisar la conexión a la base de datos para la entrada y que las queries estén correctas para el read o select, dependiendo de lo que se requiera, CRUD.

  Prueba Unitaria: Usar **Mocks** simulacros, probar que la función guardarDatos() llame al comando correcto de SQL sin necesidad de escribir en la base de datos ya que puede estar creándose en otro nodo.

  Confirmar que los datos ingresados queden guardados, la función guardar datos tenga su variable asignada.

- Acceder a su historial médico desde cualquier dispositivo con conexión a internet.

  Revisar que el código sea responsivo y se adapte a cualquier dispositivo, se probara en usabilidad, con Selenium IDE.

  Para acceder al historial revisar la función leer_usuario().
- Recibir recordatorios automatizados de citas pendientes.

  Revisar la función próximasHoras() y enviar correos de recordatorio a pacientes.
- El personal médico tendrá la capacidad de revisar y actualizar los registros de los pacientes de manera ágil y precisa.

  Entregar permisos GRANT al personal médico para modificar ficha médica.

  RBAC (Control de Acceso Basado en Roles).

  Prueba, Intentar acceder a la ficha médica con un usuario de "Paciente" y verificar que el sistema le deniegue el acceso.

- Facilitar la gestión de la información clínica y mejorar la calidad de la atención oftalmológica ofrecida por la clínica.

El uso de herramientas de automatización de pruebas mejora la eficiencia y la calidad en los procesos ágiles de desarrollo del sistema de consultas de la clínica oftalmológica, usar herramientas como **JUnit** para ejecutar la lista de verificaciones Login, CRUD, Validación, permite detectar errores en segundos cada vez que se cambia algo, asegurando que el agendamiento siempre sea eficiente y rápido.

El beneficio de la implementación de prácticas DevOps en el desarrollo del sistema de consultas oftalmológicas, el uso de **Git** para el control de versiones y **Jira** para el

seguimiento de fallas permite que el equipo de desarrollo y QA estén sincronizados, reduciendo el tiempo entre que se encuentra un error y se soluciona, permite el respaldo del código pudiendo administrar las metodologías agiles con la entrega del software.

Las estrategias específicas que pueden aplicarse para gestionar la calidad en el desarrollo del sistema de consultas oftalmológicas y garantizar la entrega de un producto de alto nivel como,

Estrategia, "saturar las probabilidades de fallo" mediante pruebas unitarias exhaustivas, Pruebas de Estrés, Métrica, la densidad de defectos, cuántos errores se encuentran por cada módulo, generalmente cuando se realizan las primeras pruebas unitarias los errores aparecen en cascada, documentar esto es importante ya que evita que nuevos errores puedan producirse, la automatización en este punto permite debuggear con mayor rapidez de forma sistémica, se utiliza la lógica de experiencia-> visibilidad-> evita error-> base de conocimientos, una vez que se debugean los errores y se depura el código los errores tienden a desaparecer y se obtiene un código listo para probar y más robusto, ahí se realiza un commit un git add -> git commit, para agregar a git, como DevOps, esta métrica dirá qué módulos son más frágiles, por ejemplo, el módulo de Historial Médico podría tener más errores iniciales que el de Login, por la falta de datos, también permite tener un mayor control ya que no olvidemos que hay prioridades para realizar las pruebas críticas.

Si la densidad de errores es muy alta en este módulo, la labor de DevOps sería no permitir que ese código avance a la siguiente etapa hasta que la métrica baje.

KPI: Tiempo promedio de resolución (MTTR): Cuánto tarda el equipo en arreglar un error reportado en Jira, para este punto el trabajo en equipo es estratégico he aprendido en estas semanas que después de la delegación de responsabilidades y las prontas entregas el equipo debe tener como métrica que cuando no se encuentra un error en un tiempo

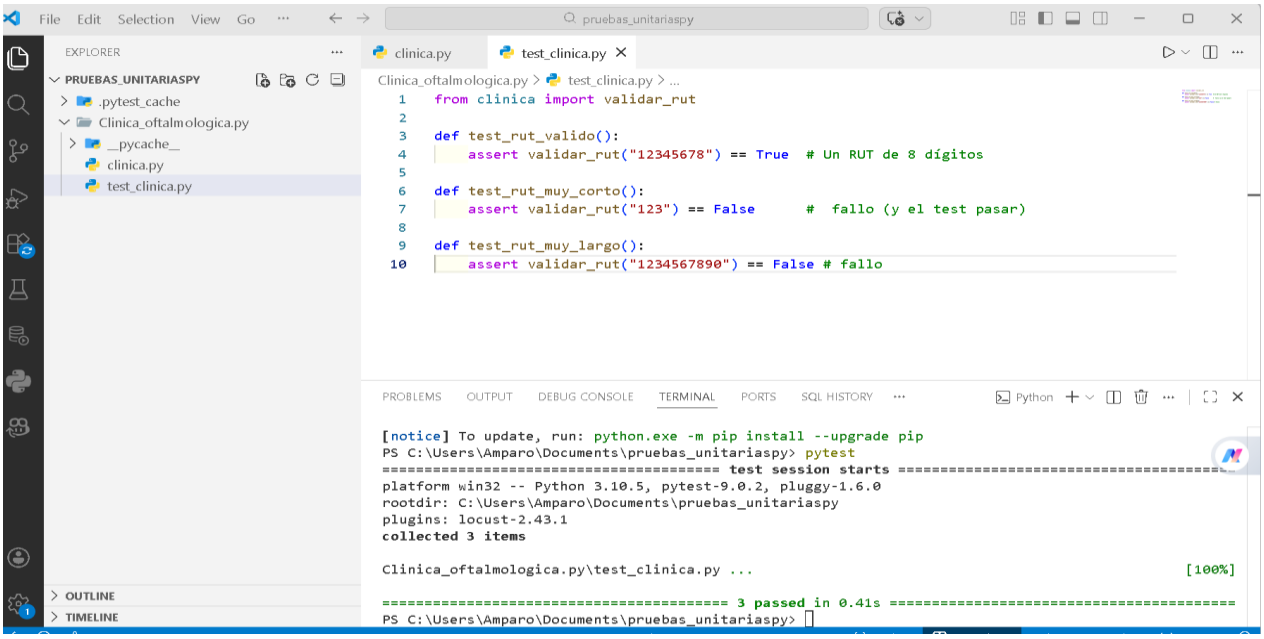
determinado es necesario compartir el suceso para que se realice una reunión especial metodología Agile para encontrar soluciones con más opiniones, enjambre o swarming, búsqueda de soluciones conjuntas.

Para realizar las pruebas unitarias Unit es java/ kotlin, si es Android se pone donde src test un trozo de código, en java sin Android debo hacer una clase para el test, ahi poner el trozo de código e importarlo, en Python desarrollo el código en Visual Studio y en consola pip install pytest igual que el Selenium web driver, se crea un archivo llamado por ejemplo, test_clinica.py, en la consola de VS Code escribir pytest y la herramienta busca todos los archivos que digan "test" y los corre de una vez.

Según el material de profundización semana 5 del ramo en curso “Al automatizar las pruebas unitarias, de integración y de aceptación, las herramientas de automatización permiten a los equipos verificar rápidamente la funcionalidad del software en cada iteración”, estas iteraciones al estar automatizadas dejan de relentizarse, debido a esto y al uso de diferentes lenguajes,

Herramientas de automatización de pruebas de sistemas,

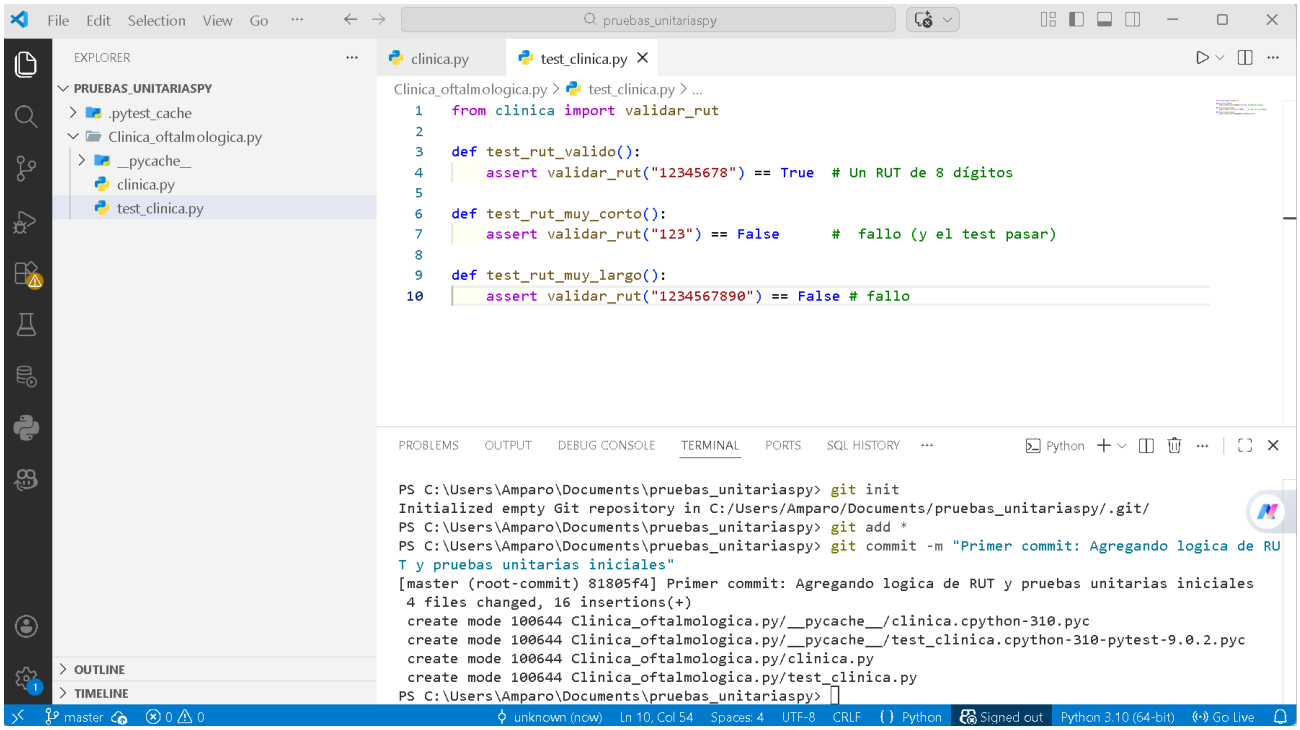
Captura de pantalla VS Code prueba automatizada unitaria pytest



Pruebas de sistema, Selenium, Cypress, TestNG.

Sistemas de control de versiones:

Git	permite trabajar offline, localmente y proporciona copias completas, permite crear ramas de forma más fácil.
SVN	Requiere conexión con el servidor, los usuarios acceden a un único repositorio.
Mercurial	permite trabajar offline y proporciona copias completas, permite crear ramas.



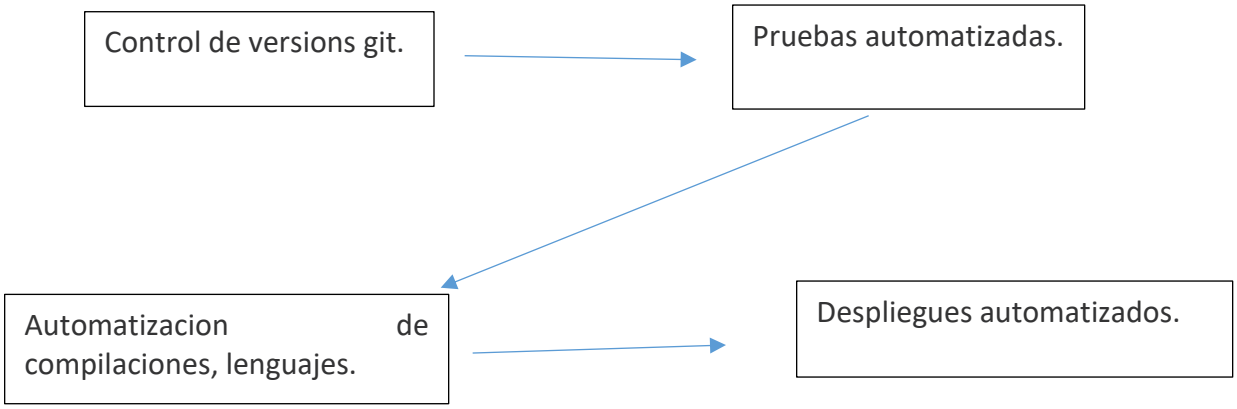
Crear una rama: git branch nueva-funcion

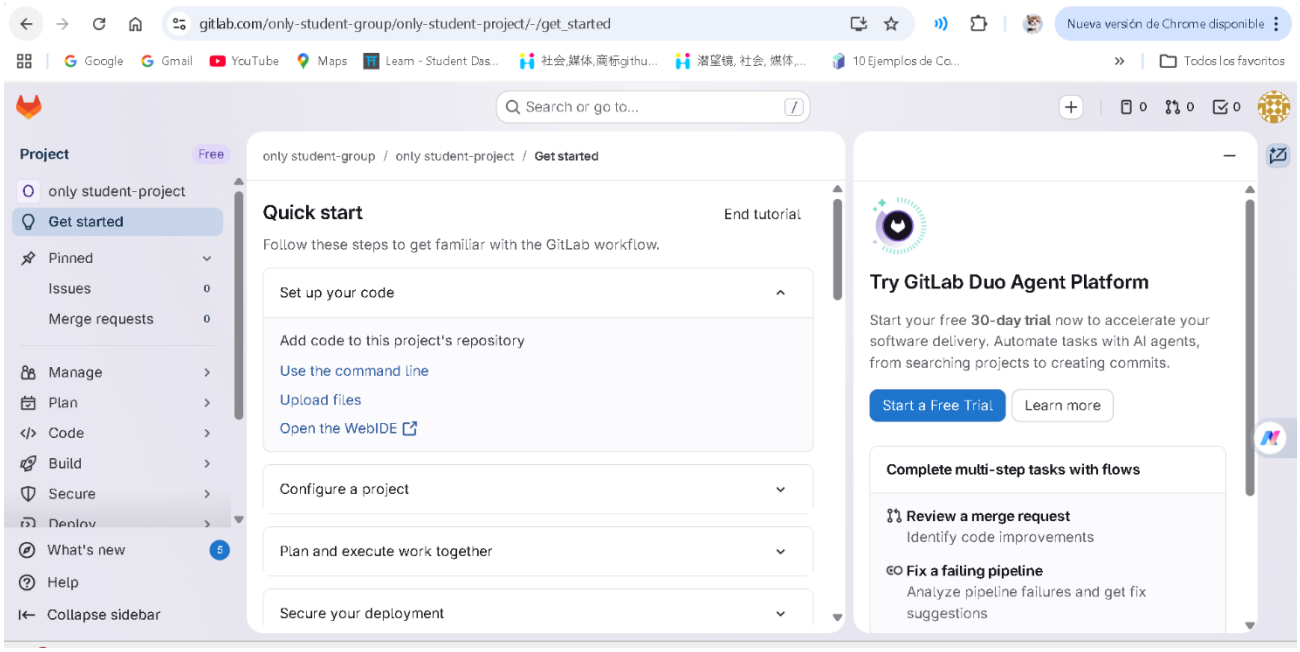
Cambiar: git checkout nueva-funcion

La rama principal (main) sigue intacta, este es un respaldo.

La Integración Continua (CI)

Según Atlassian.com es la práctica de automatizar la integración de los cambios de código de varios contribuidores en un único proyecto de software, entonces si apoyo mis fundamentos en este concepto, en un nuevo proyecto como el sistema web integral para consultas oftalmológicas, las pruebas unitarias automatizadas, junto con un control de versiones como git no solo aportan con el código funcional para realizar la integración, también, permiten que rápidamente se obtenga un feedback entre el equipo, lo que agiliza los procesos iniciales, el constante aporte de los cambios y las pruebas automatizadas permiten que las pruebas de sistema puedan realizarse con una verificación de la prueba, esta constante integración mejora la calidad del sistema en tiempo real.





Integración Continua (CI/CD): A la izquierda las opciones como **Build**, **Secure** y **Deploy**. Esto es **DevOps**, donde el código se guarda, se prueba automáticamente y se prepara para enviarlo al servidor de la clínica.

Pipeline: Abajo a la derecha en "Fix a failing pipeline", es la serie de pasos automáticos como pytest que se ejecutan cada vez que se sube código.

Según Alvarado, I., Castro Sánchez, N., Valenzuela Robles, B.,Santaolaya Salgado, R., y González Serna, J. (2024).

“En proyectos de desarrollo de software, la estimación del esfuerzo desempeña un papel crítico en el éxito o fracaso de éstos”.

Está intrínsecamente ligada al alcance, el costo y el tiempo, a pesar de la cantidad de trabajo real que requiere una tarea y el tiempo que se requiere en realizarla, después de reflexionar sobre la situación de cualquier proyecto de empresa incluso de la vida diaria,

creo que la estimación del esfuerzo va ligado a la cantidad de recursos humanos y a las capacidades de estos, sin duda la capacitación en herramientas automatizadas es critica en el manejo del desempeño, el desarrollo de las etapas de vida de un software deben manejarse con prolijidad, en este sentido las pruebas nos revelan el camino seguro validando el éxito de cada etapa, el trabajo en equipo entonces es un respaldo de recursos frente a la estimación del esfuerzo y el cumplimiento de las metas en el tiempo.

REFERENCIAS BIBLIOGRÁFICAS

Atlassian. (s. f.). ¿Qué es la integración continua?

https://www.atlassian.com/es/continuous-delivery/continuous-integration

GitLab. (s. f.). GitLab: The DevSecOps Platform.

https://about.gitlab.com/

Alvarado, I., Castro Sánchez, N., Valenzuela Robles, B.,Santaolaya Salgado, R., y González Serna, J. (2024). Los Algoritmos, métricas y validación en la estimación del esfuerzo y su impacto en la gestión de proyectos DevOps un mapeo sistemático de la literatura. ReCIBE, Revista electrónica De Computación, Informática, Biomédica Y Electrónica, 12(2), C13–15. http://recibe.cucei.udg.mx/index.php/ReCIBE/article/view/309

IACC, Evaluacion de sistemasQA, Material de profundización semana 5,

https://rise.articulate.com/share/7_qo093bV34ulZeIUs0nsdz7Jhr2kEbM#/lessons/_e8pdpNkj7F3x3RR3MDeeso_jsuvt4Vv